Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

Topologies Overview & Client-Server

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

We are looking at these topics today:

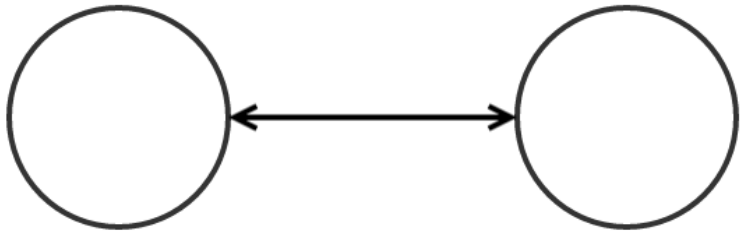
- Distributed System Topologies
- The Client/Server Architecture
- Communication Technologies
- Sockets
- Client/Server Sockets in .NET

UNIVERSITY of HULL

Distributed Architectures

- Applications and Systems are often distributed using different architectures, depending on purpose.
- We'll start today by looking at some of the physical topologies that allow distributed computing
- Physical topologies are interested in the actual physical links between different devices (or even links within a single device) and the role of each device connected by these links. Depending on the layout of these links and devices, different options are available to our distributed system.

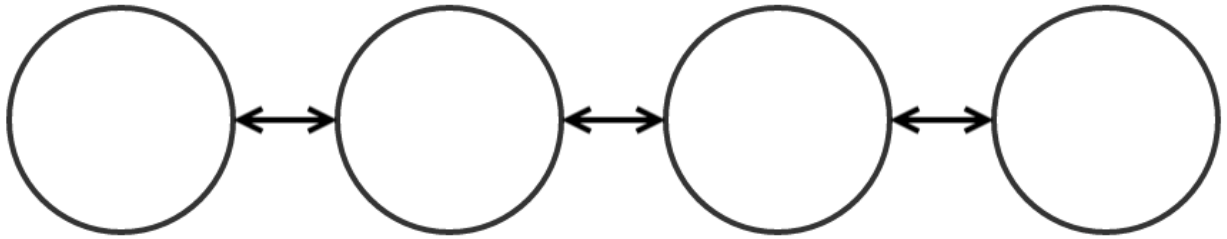
Topology 1: Single Peer-to-Peer connection



- A point-to-point network provides a single, unimpeded route from the originator to the recipient
- This design usually has two main forms: static and dynamic
  - A static point-to-point network connects the two points indefinitely,
  - The dynamic alternative can create a new, uninterrupted link for a short time, until no longer needed.
- Some network designers consider each link to be a point-to-point connection, providing a larger connectivity by assembling many point-to-point networks together

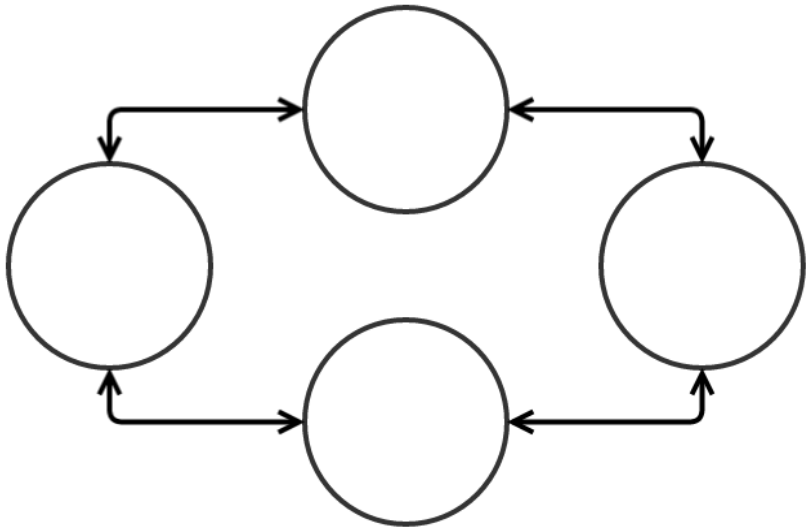
UNIVERSITY of HULL

Topology 2: Daisy Chain



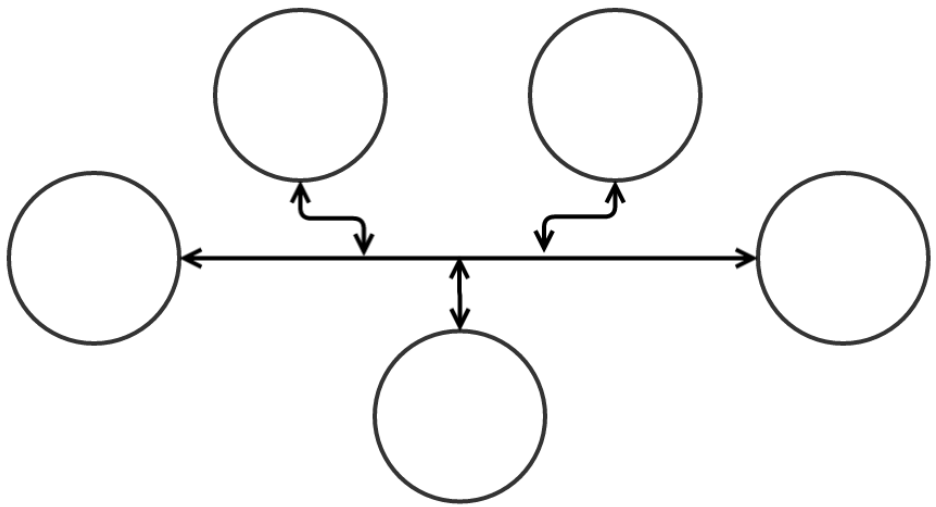
- The daisy chain or linear topology provides a series-linked set of connections.
- To send data from one end of the chain to a recipient at the other, a daisy chain network forces the data to be sent through each of the devices along the route.
- With the exception of each end of the chain, each device is connected to two other devices on either side of it.
- Should any link be broken, a connection no longer exists between the two endpoints and thus data is no longer able to flow from one to the other.

Topology 3: Circle



- Connect the ends of the daisy chain network to create a circle, or ring network
- The major benefit of a circle network is that data can now be bi-directionally routed and a single link failure doesn't destroy the network

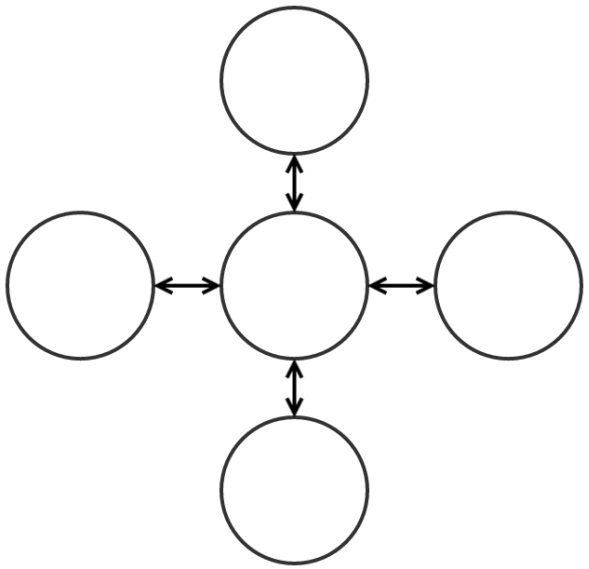
Topology 4: Bus



- Typically consists of a point-to-point network, the link of which acts as a common transmission medium for messages to be sent to devices connected along the route.
- In many implementations of this network, each transmission of data is intercepted by all machines connected to the bus, with only the intended recipient accepting the data intended for it.
- In a distributed bus network, the trunk can have many branches, giving the common transmission medium multiple endpoints.

UNIVERSITY of HULL

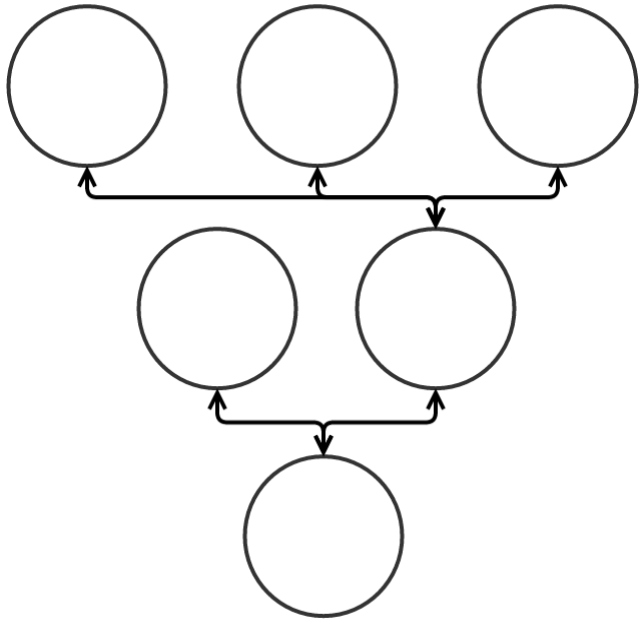
Topology 5: Star



- Uses a single machine as a central data router, with devices connected, via point-to-point links, exclusively to this router.
- To transmit from one endpoint to another, the data must pass through this central device, but the message will then be passed directly to the intended recipient.
- This topology is often used in routers and access points, where each connected device must send data through the router to access any other machine on the network.
- Central device is key!

UNIVERSITY of HULL

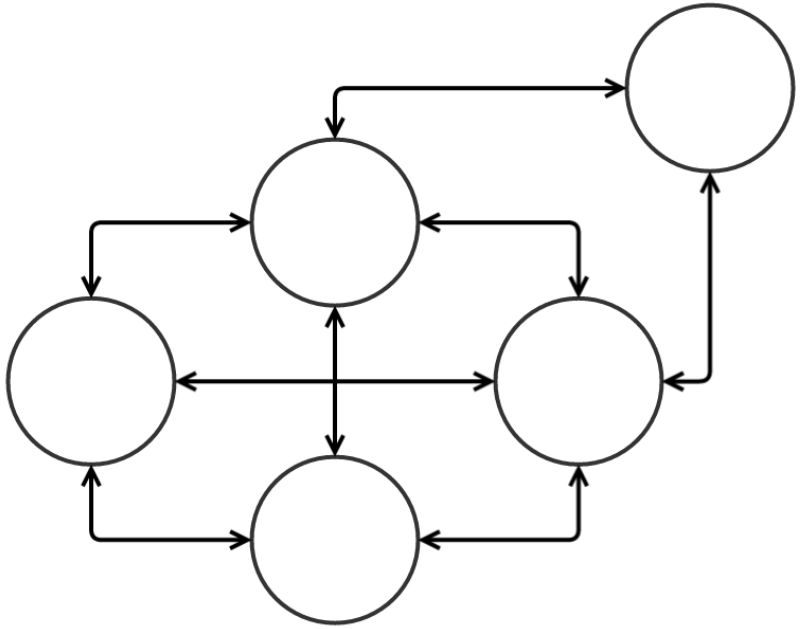
Topology 6: Tree



- A distributed hierarchical star topology.
- In a tree network, the root node operates like the central router in the star, but each connected device may also operate as a hub with additional connected devices.
- Endpoints at the network extremities are often referred to as leaf nodes and, in many real-world examples of systems employing tree topologies, are the devices which produce and consume data.

UNIVERSITY of HULL

Topology 7: Mesh



- A many-to-many connection arrangement
- Each node may be connected to a number of other devices.
- A fully connected mesh network obliges each node to be connected to all other nodes in the network
- A partially connected mesh allows some nodes to not be fully connected.
- High level of redundancy

UNIVERSITY of HULL

Topology Complexity

- Sometimes it's not that simple

- Ad-hoc networks
  - The basic principle behind an ad-hoc network is that the topology is unknown
  - As the network is deployed it may form into any topology
  - This is true of many networks, especially distributed sensor networks
  - Protocols must be devised to manage the uncertainty
- Mobile Ad-hoc Networking (MANETs)
  - A specialised form of ad-hoc networking
  - In this case, some or all of the network devices also move
  - This causes the topology to be regularly changing
  - Protocols must manage handover and redistribution
  - Think mobile phones and communication between self-driving cars

Client-Server vs Peer-Peer

- Client server may appear peer-peer through transparency…
  - Although REALLY, it is often not…

UNIVERSITY of HULL

Question

- So which real-world topology is a Client-Server Architecture?

Question

- So which real-world topology is a Client-Server Architecture?

- Answer: It could, theoretically, be any real world topology.

  However, it depends on who we are as to what part of the topology we are interested in.

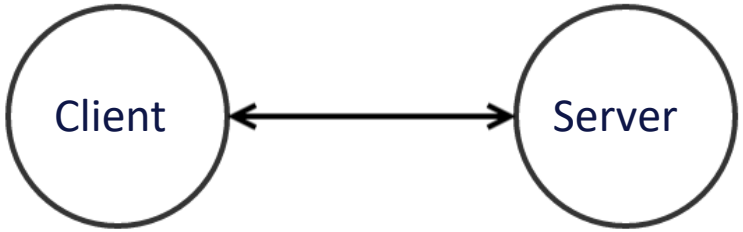
- The topology of the Internet is very complex. When making a Client-Server communication we can never be entirely sure how our requests and responses will be sent from device to device. But as long as they get there safely we should be happy!

- Therefore, as developers, we can restrict our distributed topology to a limited view, concentrating only on what is relevant to our system.

UNIVERSITY of HULL

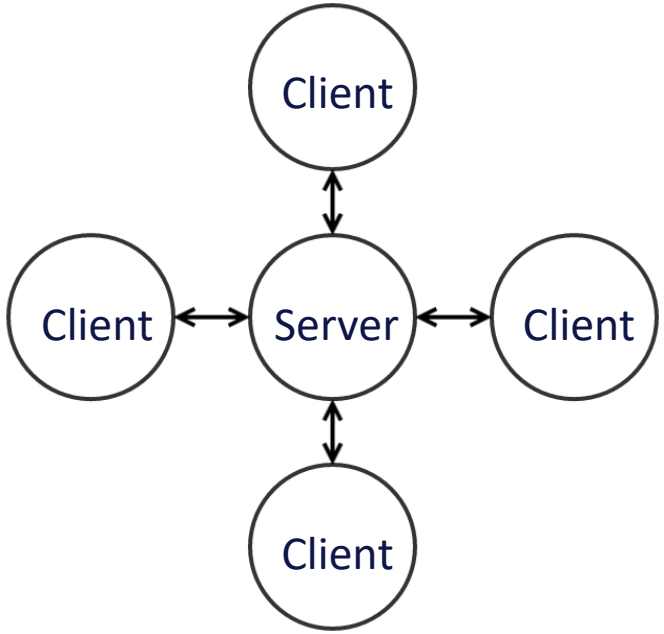
Client-Server from the Client's View

- The client is only interested in a 1-1 connection with the server
- Even if it is using the server to correspond with other clients, it is primarily only concerned with the server because it is the server that provides an intermediary (it cannot connect directly to another client).



Client-Server from the Server's View

- The server often has to manage a number of connections from different clients.
- Normally, this topology is considered as a star.
- This means that concurrency in the server is (usually) absolutely key to it working properly.



UNIVERSITY of HULL

Client-Server Architectures

- Thick clients
  - Client: application instance
  - Server: data transmission
  - Each client runs an instance of the application
  - Responsible for data objects

- Thin clients
  - Client: usually user interface only
  - Server: data processing and object manipulation
  - Server runs the computationally intensive operations
  - Data objects processed usually on the server

7 Layers of the OSI Model

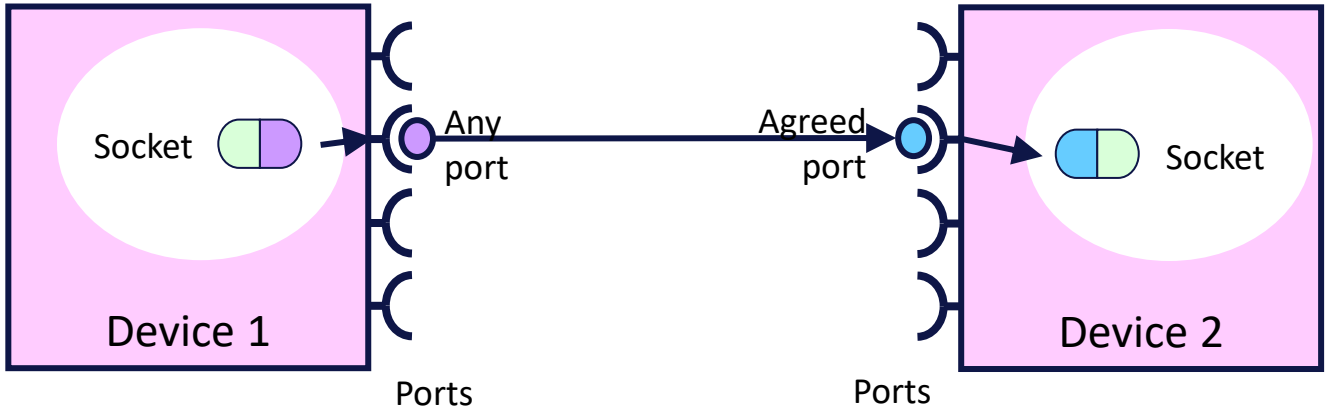
Developer	7	Application	• End User Layer • HTTP,FTP,IRC,SSH,DNS,SMTP	Services that are used by the end user
Developer	6	Presentation	• Syntax Layer • SSL,SSH,IMAP,FTP,MPEG,JPEG	Translates, Encrypts/Decrypts & compress the data
OS	5	Session	• Sync & send to port • APIs,Socket,WinSock	Creates, manages & terminates session
OS	4	Transport	• End to End communication • TCP & UDP	Manages the delivery & checking error of packets
Hardware/Firmware	3	Network	• Packets • IP,ICMP,IPSec,IGMP	Decides the shortest Path among routes to transmit data
Hardware/Firmware	2	Data Link	• Frames • Ethernet,PPP,Switch,Bridge	Data packed into frames are shared between nodes
Hardware/Firmware	1	Physical	• Physical structure • Fiber, Wireless, Hubs, Repeaters	Physical medium where data is transmitted in form of binary (0s & 1s)

Making the Connection **(SESSION LAYER)**

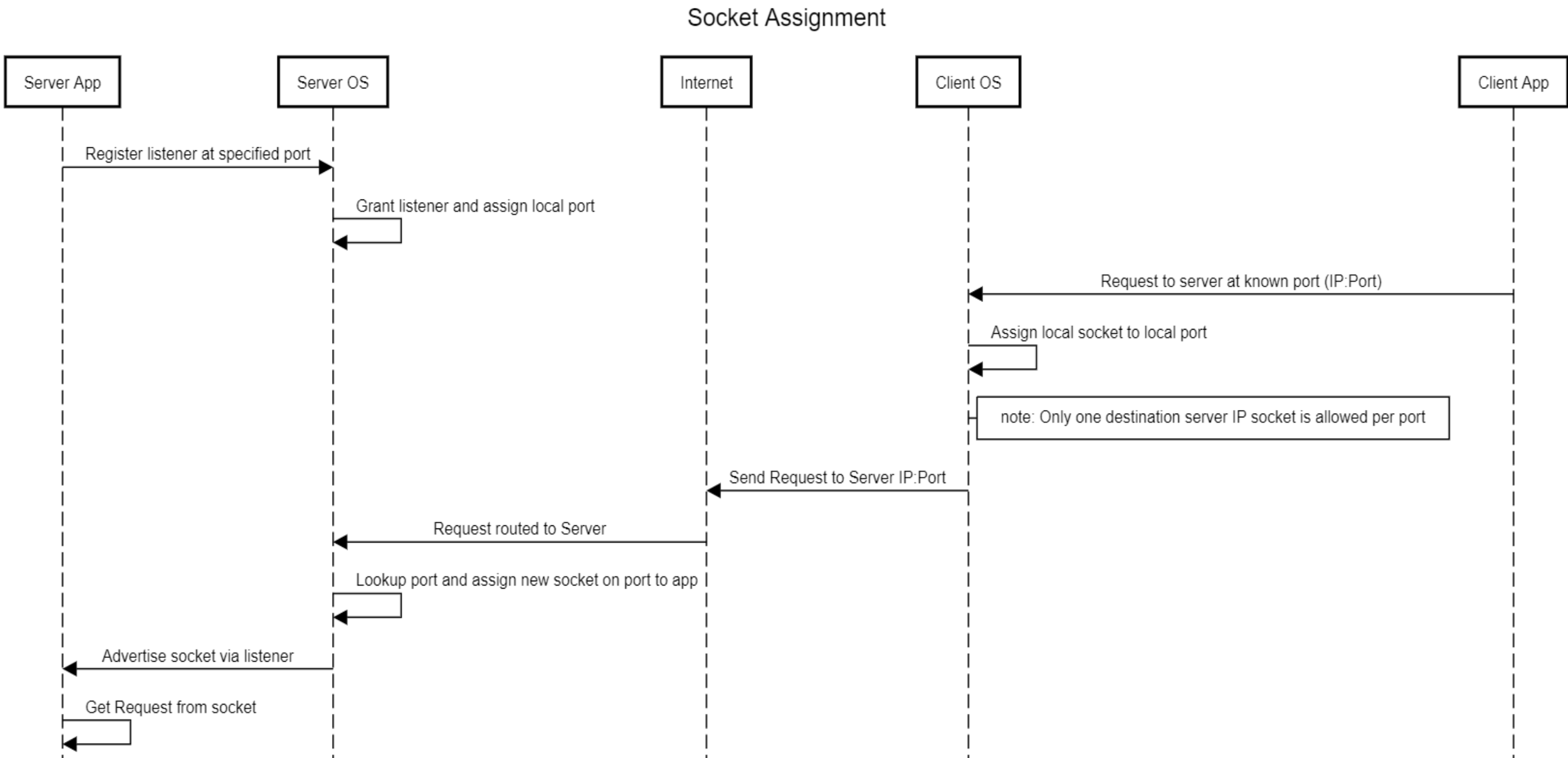
A communication channel is created by connecting two sockets (appears peer-to-peer)

- An IP address is used to identify each connected device
- IPv4: 10.20.30.40
  - We are running out (/have run out) of IP addresses in this format so we are now using IPv6
- A socket is assigned by asking for a particular port (e.g. port 80)
  - Format: 10.20.30.40:80
- A bidirectional socket is then assigned:

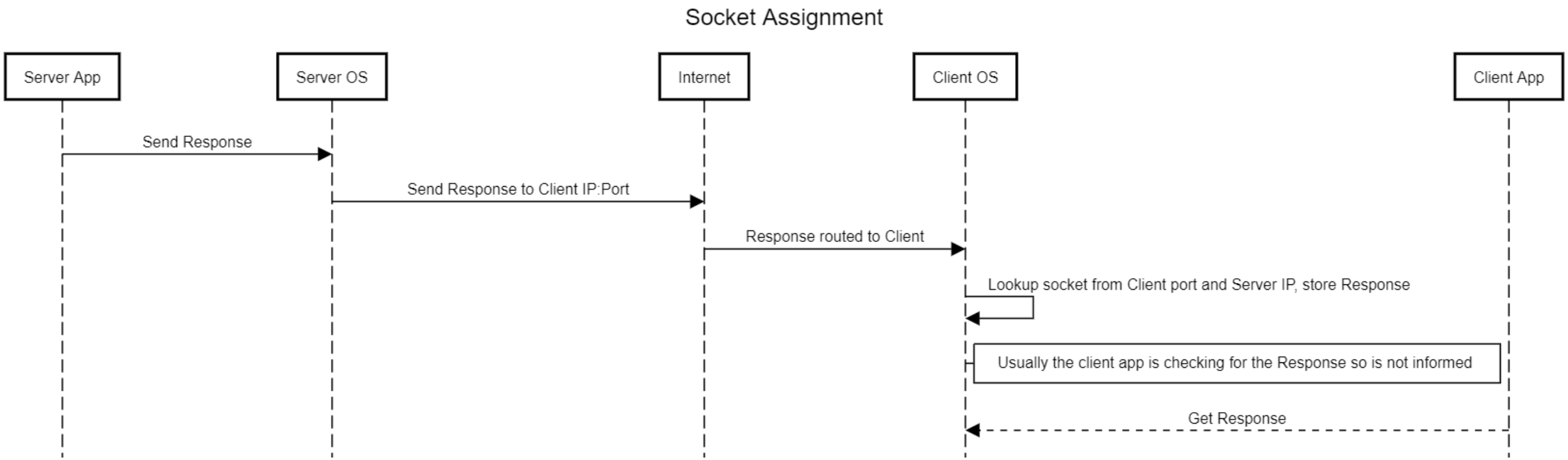
A socket is **completely virtual**. It is not a physical connection but a logical connection between two machines formed by the OS using the underlying protocols and software to do the real heavy lifting.



Making the Connection



Making the Connection



UNIVERSITY of HULL

Sockets

- Platform-independent
  - Can transport any kind of data
  - Support many protocols e.g. TCP, UDP
- Low-level technology
  - Transmission on the level of bytes
  - Requires reimplementation of some form of custom application protocol
    - Perhaps not as good as a standardised protocol…
  - Not suitable for high-level applications

Client Socket in .NET

```
TcpClient tcpClient = new TcpClient();

tcpClient.Connect(hostname, port);

NetworkStream nStream = tcpClient.GetStream();
```

- Constructing a new TcpClient instance
  - A reference to System.Net.Sockets must be added to the project
- Constructor arguments
  - hostname: computer / domain name or IP address
  - port: chosen communication port

UNIVERSITY of HULL

Server Socket in .NET

```
int port = 5002;
string address = "127.0.0.1";

TcpListener tcpListener = new TcpListener(IPAddress.Parse(address), port);
tcpListener.Start();

TcpClient tcpClient = tcpListener.AcceptTcpClient();
NetworkStream nStream = tcpClient.GetStream();
```

- A server socket is a special type of socket which
  - waits for and accepts incoming connections
  - returns a **new** socket instance which is actually used to send/receive data
- TcpListener
  - has AcceptTcpClient() method which we call to wait for incoming connections
  - is a server socket implementation provided by .NET

UNIVERSITY of HULL

Server Sockets

- If we are expecting more than one simultaneous client request…
  - Once an incoming connection has been established
  - The **new**ly created socket should be passed to a separate (unique) thread to interact with the client
    - So that the server can continue to listen
    - And multiple clients can communicate with the server at the same time
  - Once the connection is no longer required the unique thread can be terminated
    - To free up resources

- If we are only ever expecting one client request at a time we can ignore multithreading
  - But this is an unusual occurrence in distributed applications

Normal Distributed Programming

- You may have used sockets previously (and should have seen them in the workshop)
- It's worth pointing out that developers rarely write distributed systems/applications by writing Socket code any more.
- Why not?

Normal Distributed Programming

- There are lots of frameworks which manage this for you.

  ‘Don’t reinvent the wheel’

Most developers want the same things so it’s not worth the time for everyone to try to create their own versions of the same solution.

Instead, we use standardised, common tools which have been well tested and will be cross-compatible with other products that have already been created.

We’ll investigate some of these in lectures and in the labs.

Common Communication Technologies

Sockets Examples:

- TCP / UDP (Transmission Control Protocol / User Datagram Protocol)

Web Services Protocol/Architecture Examples:

- SOAP (Simple Object Access Protocol)
- RSS (Really Simple Syndication)
- REST (REpresentational State Transfer)
- WSDL (Web Services Description Language)
- WADL (Web Application Description Language)

Technology Frameworks Examples:

- Windows Communication Foundation
- ASP.NET
- Ruby on Rails
- gSOAP

Remote Procedure Calling (RPC):

- .NET Remoting (deprecated)
- gRPC (using HTTP/2)
- SignalR (using WebSockets)

UNIVERSITY of HULL

Reflection

We've looked at:

- Distributed System Topologies
- The Client/Server Architecture
- Communication Technologies
- Sockets
- Client/Server Sockets in .NET

In the next session we will look at Design Patterns, explicitly message patterns

UNIVERSITY of HULL